MATRIX OPERATION SYSTEM

Cornerstone project work submitted to Aditya University in partial fulfilment of the requirements for the award of the Degree of

B.Tech. degree

in

Computer Science & Engineering (2026-2027)

SUBMITTED BY

TAPAS DAS	**25B11AIB77**
SOURAV DEV	**25B11AI322**
SUSHMITA ROY	**25B11AIB54**

UNDER THE GUIDANCE OF

Ms.T.Vijaya Lakshmi

Assistant Professor

Department of AIML



Department of Computer Science & Engineering
Aditya University, Aditya Nagar, ADB Road,
Surampalem, Kakinada, District, Andhra Pradesh, India - 533437

DECLARATION

I hereby declare that the Cornerstone project work entitled **“Matrix Operation System”** is being submitted by me for the award of the B. Tech Degree in Computer Science and Engineering, Aditya University, Aditya Nagar, ADB Road, Surampalem is a record of original work carried out by us. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree or diploma in this or any other institution. We also confirm that this work has been carried out with academic honesty and complies with the Institutional Academic Integrity and AI Usage Policy.

Submitted By

TAPAS DAS	25B11AIB77
SOURAV DEV	25B11AI322
SUSHMITA ROY	25B11AIB54

Place:Surampalem

Date:29.09.2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ADITYA UNIVERSITY, ADITYA NAGAR,

SURAMPALEM.



This is to certify that the cornerstone project entitled **"MATRIX OPERATION SYSTEM"** is being submitted by Tapas Das, Sourav Dev & Sushmita Roy in partial fulfilment for the award of B. Tech Degree from Aditya University, is a record of original work done by them under my supervision. The contents of this project, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Project Guide

Ms.T.Vijaya Lakshmi

Assistant Professor

Department of CSE

Aditya University

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Ms.T.Vijaya Lakshmi, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our **Dr. Bollu Manikyala Rao, Associate Professor and Dr. Dasari Veeraswamy, Assistant Professor,** Department of AIML,for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, **Dr. Kovvuri N Bhargavi, Associate Professor & HOD**, for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **DR. M.V. RAJESH, ASSOCIATE DEAN, SCHOOL OF COMPUTING**, for his encouragement and support during our project.

We would like to express our sincere gratitude to **DR. G. SURESH, REGISTRAR; DR. A. RAMESH, PRO VICE-CHANCELLOR (ENGINEERING & SCIENCES); DR. S. RAMA SREE, PRO VICE-CHANCELLOR (ACADEMICS); DR. M.B. SRINIVAS, VICE-CHANCELLOR; AND DR. M. SREENIVASA REDDY, DEPUTY PRO CHANCELLOR AND MANAGEMENT, ADITYA UNIVERSITY**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **FACULTY MEMBERS, LAB TECHNICIANS, NON-TEACHING STAFF, AND OUR FRIENDS** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Name of the Students: TAPAS DAS, SOURAV DEV & SUSHMITA ROY
Date:29.09.2026

ABSTRACT

Matrices are fundamental mathematical structures widely used in fields such as mathematics, computer science, engineering, graphics, and artificial intelligence. Manual matrix calculations become increasingly lengthy, repetitive, and difficult to verify as matrix dimensions grow. To address this, the **Matrix Operation System** is proposed as a C++ based application that provides a structured, menu-driven interface for users to input matrices and perform standard matrix calculations.

The system is designed with a strong emphasis on Object-Oriented Programming (OOP) principles, including encapsulation, modular design, and reusable methods. Key features include matrix input and validation, addition, subtraction, multiplication, transpose, determinant, and inverse operations. The architecture is divided into three distinct layers: the Presentation Layer (console/menu interface), the Business Logic Layer (Matrix Manager, Operation Manager, Validation Manager), and the Data/Object Layer (Matrix Objects, 2D Arrays/Vectors, Result Data). The system validates matrix dimensions before performing any operation, ensuring accuracy and reducing the potential for manual errors. This documentation details the project's planning, design, implementation, and expected outcomes.

ABBREVATIONS

- **OOP** – Object-Oriented Programming
- **C++** – C Plus Plus (Programming Language)
- **ER** – Entity-Relationship
- **UI** – User Interface
- **AIML** – Artificial Intelligence and Machine Learning
- **CSE** – Computer Science and Engineering
- **PBL** – Project-Based Learning
- **2D** – Two Dimensional
- **A.columns** – Columns of Matrix A
- **B.rows** – Rows of Matrix B

LIST OF FIGURES

1. **Fig 1.1:** Architecture Diagram of Matrix Operation System
2. **Fig 1.2:** ER Diagram of Matrix Operation System
3. **Fig 1.3:** Planning and Timeline Diagram (Gantt Chart)
4. **Fig 2.1:** Output Screens of Matrix Operations

LIST OF TABLES

1. **Table 1.1:** Planned Modules and Functionalities
2. **Table 1.2:** Role of Each Individual

TABLE OF CONTENTS

CONTENTS	PAGE NO
Title Page	I
Certificate	II
Declaration	III
Acknowledgement	IV
Abstract	V
Abbreviations	VI
List of Figures	VII
List of Tables	VIII
CHAPTER -1 **Project Planning**	1
1.1. Problem statement	2
1.2. Project Objective	3
1.3. Planned Modules	4
1.4. System Architecture (Algorithm & Flowchart)	5
CHAPTER -1 **Project Implementation**	
2.1. OOP Concepts Used for implementation	6
2.2. Code	7
2.3. Output Screens	10
CHAPTER-3 **Conclusion & Future Scope**	
3.1. Conclusion	12
3.2. Future Scope	12

CHAPTER -1 Project Planning

1.1. Problem Statement

Manual matrix calculations are repetitive, time-consuming, and prone to human error. As the size of the matrix increases, the complexity and difficulty of verifying results manually also increase significantly. Users, particularly students, faculty, and developers, require a simple, reliable, and efficient C++ system that can perform multiple matrix operations accurately while checking for valid dimensions before calculation. There is a need for a consolidated tool that reduces manual effort, ensures mathematical accuracy, and provides a clear, structured way to perform these operations.

1.2. Project Objective

The primary objectives of the Matrix Operation System are:

- To perform common matrix operations such as addition, subtraction, multiplication, transpose, determinant, and inverse.
- To validate matrix dimensions before executing any calculation to ensure compatibility.
- To reduce manual effort and minimize errors associated with repetitive calculations.
- To apply Object-Oriented Programming (OOP) concepts in C++ for a clean, modular, and reusable design.
- To display results clearly through a user-friendly, menu-driven interface.
- To keep modules reusable and extendable for future enhancements.

1.3. Planned Modules

The system is divided into six key modules to ensure a structured and modular development process.

Module	Description
Matrix Class	Stores rows, columns, and elements of a matrix. Serves as the foundational data structure.
Input & Validation	Accepts matrix input from the user and checks for valid dimensions before allowing operations.
Basic Operations	Handles addition and subtraction for matrices that are compatible in size.

Multiplication	Multiplies two matrices only when the number of columns of the first matrix equals the number of rows of the second matrix (A.columns = B.rows).
Advanced Operations	Computes the transpose, determinant, and inverse of a matrix.
Menu / Result	Displays a menu for the user to select an operation and presents the final output clearly.

Table 1.1: Planned Modules and Functionalities

1.4. System Architecture (Algorithm & Flowchart)

The system architecture is organized into three distinct layers, ensuring a separation of concerns and modular design.

1. Presentation Layer:

- **Console / Menu-driven User Interface**
- **Functions:** Input Matrix, Select Operation, Display Result.

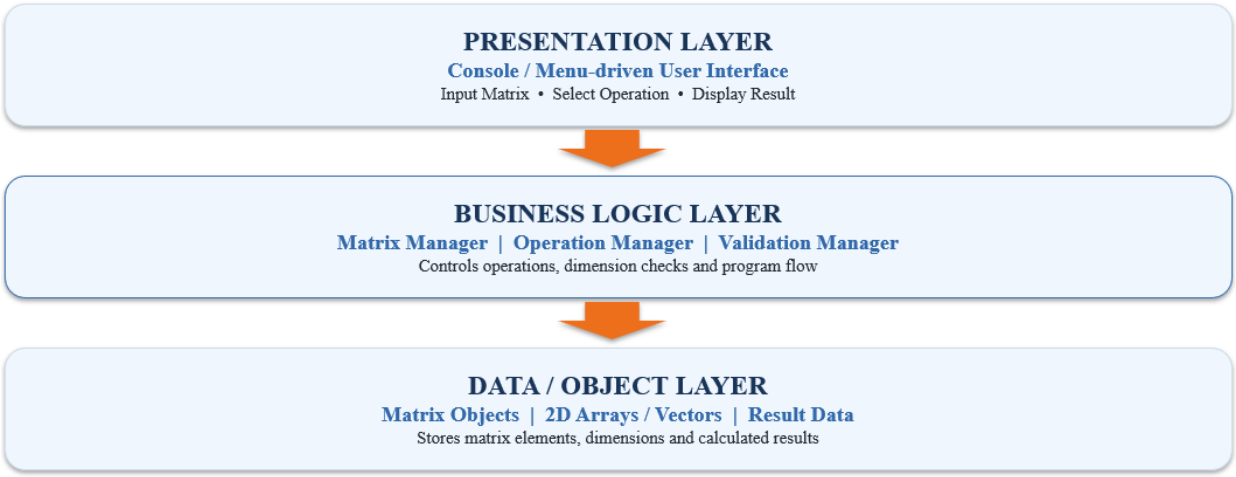
2. Business Logic Layer:

- **Components:** Matrix Manager, Operation Manager, Validation Manager.
- **Functions:** Controls operations, dimension checks, and program flow.

3. Data / Object Layer:

- **Components:** Matrix Objects, 2D Arrays/Vectors, Result Data.
- **Functions:** Stores matrix elements, dimensions, and calculated results.

Fig 1.1: Architecture Diagram of Matrix Operation System



Algorithm (General Flow):

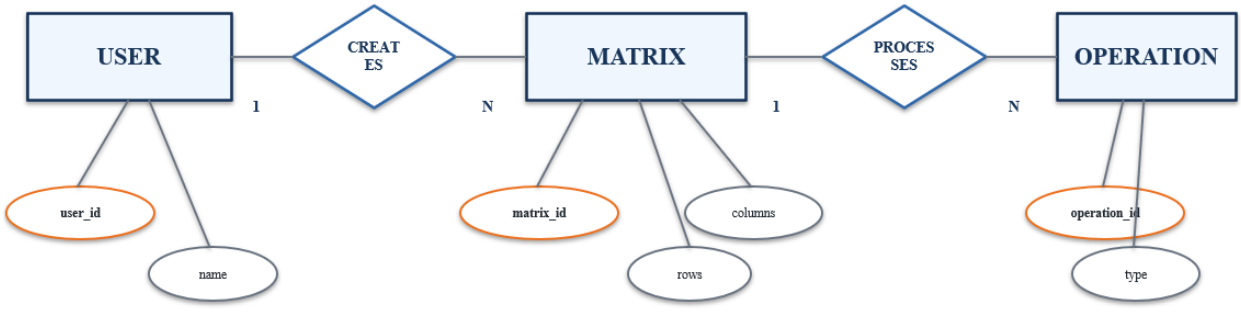
1. Start the program.
2. Display the main menu.
3. Accept user choice for the desired matrix operation.
4. Based on the choice, prompt the user to enter the dimensions and elements of the required matrix/matrices.
5. Validate dimensions for the selected operation (e.g., addition requires equal dimensions; multiplication requires A.columns = B.rows).
6. If dimensions are invalid, display an error message and return to the menu.
7. If dimensions are valid, perform the selected operation using the Matrix class methods.
8. Display the result.
9. Ask the user if they wish to perform another operation. If yes, return to the menu; if no, exit the program.
10. Stop.

ER Diagram:

The Entity-Relationship (ER) diagram defines the data model. A **USER** creates a **MATRIX**, which is processed by an **OPERATION** to produce a result.

- **USER:** Attributes include user_id and name.
- **MATRIX:** Attributes include matrix_id, rows, and columns.
- **OPERATION:** Attributes include operation_id and type.
- **Relationships:** A user creates one or more matrices (1:N). Each matrix is processed by one or more operations (1:N).

Fig 1.2: ER Diagram of Matrix Operation System



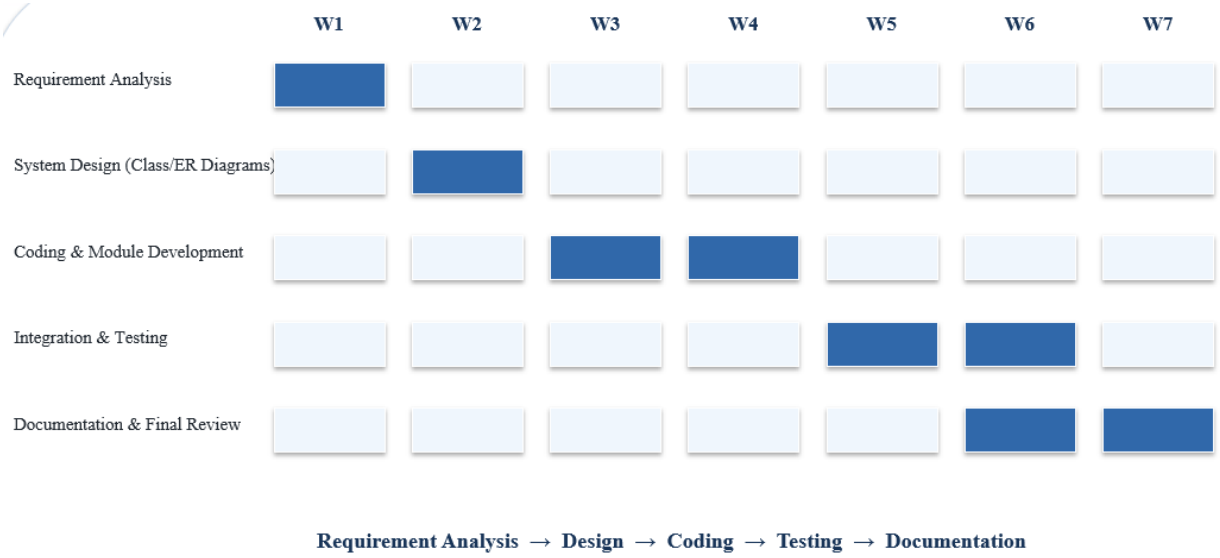
USER → creates MATRIX → processed by OPERATION → RESULT

Planning and Timeline Diagram:

The project follows a structured timeline over seven weeks:

- **W1:** Requirement Analysis
- **W2:** System Design (Class/ER Diagrams)
- **W3-W4:** Coding & Module Development
- **W5-W6:** Integration & Testing
- **W6-W7:** Documentation & Final Review

Fig 1.3: Planning and Timeline Diagram (Gantt Chart)



Role of Each Individual:

Team Member	Role
Sourav Dev	Matrix class design, constructors, and input handling.
Sushmita Roy	Addition, subtraction, and multiplication modules.
Tapas Das	Determinant, inverse, validation, testing, and debugging.
Niloy Mallick	UI/menu design, diagrams, documentation, and presentation.

Table 1.2: Role of Each Individual

2.1. OOP Concepts Used for Implementation

The implementation of the Matrix Operation System heavily relies on core Object-Oriented Programming (OOP) concepts in C++ to ensure the code is robust, reusable, and maintainable.

- **Encapsulation:** The Matrix class encapsulates the data (rows, columns, and a 2D array/vector of elements) and the methods that operate on that data (input, display, add, multiply, etc.). This protects the internal state of the matrix from unauthorized access and modification.
- **Modularity:** The system is broken down into smaller, independent modules (as listed in Section 1.3). Each module is responsible for a specific functionality, making the code easier to understand, test, and debug.
- **Reusability:** Methods defined in the Matrix class (e.g., getRows(), getColumns(), getElement()) are reused across different operations. This reduces code duplication and improves efficiency.
- **Function-based Operation Handling:** The system uses functions and member functions to handle different operations. The main menu directs the program flow to the appropriate function, which then utilizes the Matrix class methods to perform the calculation.

2.2. Code

```
#include <iostream>
#include <vector>
#include <iomanip>
#include <cmath>
using namespace std;

// ============================================================
//                  MATRIX CLASS DEFINITION
// ============================================================
class Matrix {
private:
    int rows, cols;
    vector<vector<double> > data;
```

```
public:
    // ---------- Constructor ----------
    Matrix(int r, int c) {
        rows = r;
        cols = c;
        data.resize(rows, vector<double>(cols, 0.0));
    }

    // ---------- Default Constructor ----------
    Matrix() {
        rows = 0;
        cols = 0;
    }

    // ---------- Input Method ----------
    void input(string name) {
        cout << "Enter elements of Matrix " << name << ":" << endl;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                cout << "Element [" << i << "][" << j << "]: ";
                cin >> data[i][j];
            }
        }
    }

    // ---------- Display Method ----------
    void display(string name) {
        cout << name << ":" << endl;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                cout << setw(8) << fixed << setprecision(2) << data[i][j] << "  ";
```

```
        }
        cout << endl;
    }
}

// ---------- Getter Methods ----------
int getRows() { return rows; }
int getCols() { return cols; }
double getElement(int i, int j) { return data[i][j]; }
void setElement(int i, int j, double value) { data[i][j] = value; }

// ---------- Addition ----------
Matrix add(Matrix &other) {
    Matrix result(rows, cols);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            result.data[i][j] = data[i][j] + other.data[i][j];
        }
    }
    return result;
}

// ---------- Subtraction ----------
Matrix subtract(Matrix &other) {
    Matrix result(rows, cols);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            result.data[i][j] = data[i][j] - other.data[i][j];
        }
    }
    return result;
```

```
}

// ---------- Multiplication ----------
Matrix multiply(Matrix &other) {
    Matrix result(rows, other.cols);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < other.cols; j++) {
            result.data[i][j] = 0;
            for (int k = 0; k < cols; k++) {
                result.data[i][j] += data[i][k] * other.data[k][j];
            }
        }
    }
    return result;
}

// ---------- Transpose ----------
Matrix transpose() {
    Matrix result(cols, rows);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            result.data[j][i] = data[i][j];
        }
    }
    return result;
}

// ---------- Get SubMatrix (for determinant/inverse) ----------
Matrix getSubMatrix(int r, int c) {
    Matrix sub(rows - 1, cols - 1);
    int subi = 0;
```

```
    for (int i = 0; i < rows; i++) {
        if (i == r) continue;
        int subj = 0;
        for (int j = 0; j < cols; j++) {
            if (j == c) continue;
            sub.data[subi][subj] = data[i][j];
            subj++;
        }
        subi++;
    }
    return sub;
}

// ---------- Determinant (Recursive) ----------
double determinant() {
    if (rows == 1 && cols == 1) {
        return data[0][0];
    }
    if (rows == 2 && cols == 2) {
        return (data[0][0] * data[1][1]) - (data[0][1] * data[1][0]);
    }

    double det = 0;
    int sign = 1;
    for (int j = 0; j < cols; j++) {
        Matrix sub = getSubMatrix(0, j);
        det += sign * data[0][j] * sub.determinant();
        sign = -sign;
    }
    return det;
}
```

```
// ---------- Cofactor ----------
double cofactor(int i, int j) {
    Matrix sub = getSubMatrix(i, j);
    double sign = ((i + j) % 2 == 0) ? 1 : -1;
    return sign * sub.determinant();
}

// ---------- Adjoint ----------
Matrix adjoint() {
    Matrix adj(rows, cols);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            adj.data[j][i] = cofactor(i, j);
        }
    }
    return adj;
}

// ---------- Inverse ----------
Matrix inverse() {
    double det = determinant();
    if (det == 0) {
        cout << "\nERROR: Inverse does not exist (Determinant = 0)!" << endl;
        Matrix empty(0, 0);
        return empty;
    }
    Matrix adj = adjoint();
    Matrix inv(rows, cols);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
```

```
            inv.data[i][j] = adj.data[i][j] / det;
        }
    }
    return inv;
  }
};

// ============================================================
//                  VALIDATION FUNCTIONS
// ============================================================
bool validateSameSize(Matrix &A, Matrix &B) {
  return (A.getRows() == B.getRows() && A.getCols() == B.getCols());
}

bool validateMultiplication(Matrix &A, Matrix &B) {
  return (A.getCols() == B.getRows());
}

bool validateSquare(Matrix &A) {
  return (A.getRows() == A.getCols());
}

// ============================================================
//                  DISPLAY MENU
// ============================================================
void displayMenu() {
  cout << "\n========================================" << endl;
  cout << "      MATRIX OPERATION SYSTEM          " << endl;
  cout << "========================================" << endl;
  cout << "1. Add Matrices" << endl;
  cout << "2. Subtract Matrices" << endl;
```

```
    cout << "3. Multiply Matrices" << endl;
    cout << "4. Transpose Matrix" << endl;
    cout << "5. Determinant of Matrix" << endl;
    cout << "6. Inverse of Matrix" << endl;
    cout << "7. Exit" << endl;
    cout << "========================================" << endl;
    cout << "Enter your choice (1-7): ";
}

// ============================================================
//                  EXIT MESSAGE
// ============================================================
void displayExitMessage() {
    cout << "\n========================================" << endl;
    cout << "   Thank you for using Matrix Operation System!" << endl;
    cout << "              Exiting..." << endl;
    cout << "========================================" << endl;
}

// ============================================================
//                  MAIN FUNCTION
// ============================================================
int main() {
    int choice;

    do {
        displayMenu();
        cin >> choice;

        switch (choice) {
```

```
// ---------------- ADDITION ----------------
case 1: {
    int r, c;
    cout << "\nEnter number of rows: ";
    cin >> r;
    cout << "Enter number of columns: ";
    cin >> c;

    Matrix A(r, c), B(r, c);
    A.input("A");
    cout << endl;
    B.input("B");

    if (!validateSameSize(A, B)) {
        cout << "\nERROR: Matrix dimensions must be equal for addition!" << endl;
        break;
    }

    Matrix C = A.add(B);
    cout << "\nMatrix A:" << endl;
    A.display("A");
    cout << endl;
    B.display("B");
    cout << "\nResult of Addition (A + B):" << endl;
    C.display("Result");
    break;
}

// ---------------- SUBTRACTION ----------------
case 2: {
    int r, c;
```

```
    cout << "\nEnter number of rows: ";
    cin >> r;
    cout << "Enter number of columns: ";
    cin >> c;

    Matrix A(r, c), B(r, c);
    A.input("A");
    cout << endl;
    B.input("B");

    if (!validateSameSize(A, B)) {
        cout << "\nERROR: Matrix dimensions must be equal for subtraction!" << endl;
        break;
    }

    Matrix C = A.subtract(B);
    cout << "\nMatrix A:" << endl;
    A.display("A");
    cout << endl;
    B.display("B");
    cout << "\nResult of Subtraction (A - B):" << endl;
    C.display("Result");
    break;
}

// ---------- MULTIPLICATION ----------
case 3: {
    int r1, c1, r2, c2;
    cout << "\nEnter number of rows of Matrix A: ";
    cin >> r1;
    cout << "Enter number of columns of Matrix A: ";
```

```
            cin >> c1;
            cout << "Enter number of rows of Matrix B: ";
            cin >> r2;
            cout << "Enter number of columns of Matrix B: ";
            cin >> c2;

            if (c1 != r2) {
                cout << "\nERROR: Matrix dimensions are not compatible for multiplication!" <<
endl;
                cout << "(A.columns must equal B.rows)" << endl;
                cout << "A.columns (" << c1 << ") != B.rows (" << r2 << ")" << endl;
                break;
            }

            Matrix A(r1, c1), B(r2, c2);
            A.input("A");
            cout << endl;
            B.input("B");

            Matrix C = A.multiply(B);
            cout << "\nMatrix A (" << r1 << "x" << c1 << "):" << endl;
            A.display("A");
            cout << endl;
            B.display("B");
            cout << "\nResult of Multiplication (A * B):" << endl;
            C.display("Result");
            break;
        }

        // ---------------- TRANSPOSE ----------------
        case 4: {
```

```
    int r, c;
    cout << "\nEnter number of rows: ";
    cin >> r;
    cout << "Enter number of columns: ";
    cin >> c;

    Matrix A(r, c);
    A.input("A");

    cout << "\nOriginal Matrix:" << endl;
    A.display("A");

    Matrix T = A.transpose();
    cout << "\nTranspose of Matrix:" << endl;
    T.display("Transpose");
    break;
}

// ---------- DETERMINANT ----------
case 5: {
    int n;
    cout << "\nEnter size of matrix (n x n): ";
    cin >> n;

    Matrix A(n, n);
    A.input("A");

    if (!validateSquare(A)) {
        cout << "\nERROR: Determinant only defined for square matrices!" << endl;
        break;
    }
```

```
    cout << "\nMatrix A:" << endl;
    A.display("A");

    double det = A.determinant();
    cout << "\nDeterminant = " << det << endl;
    break;
}

// ---------------- INVERSE ----------------
case 6: {
    int n;
    cout << "\nEnter size of matrix (n x n): ";
    cin >> n;

    Matrix A(n, n);
    A.input("A");

    if (!validateSquare(A)) {
        cout << "\nERROR: Inverse only defined for square matrices!" << endl;
        break;
    }

    cout << "\nMatrix A:" << endl;
    A.display("A");

    double det = A.determinant();
    cout << "\nDeterminant = " << det << endl;

    if (det == 0) {
        cout << "\nERROR: Inverse does not exist (Determinant = 0)!" << endl;
```

```
            break;
        }

        Matrix inv = A.inverse();
        cout << "\nInverse of Matrix:" << endl;
        inv.display("Inverse");
        break;
    }

    // ---------- EXIT ----------
    case 7:
        displayExitMessage();
        break;

    // ---------- INVALID ----------
    default:
        cout << "\nInvalid choice! Please enter a number between 1 and 7." << endl;
    }

  } while (choice != 7);

  return 0;
}
```

2.3. Output Screens

Fig 2.1: Main Menu Display

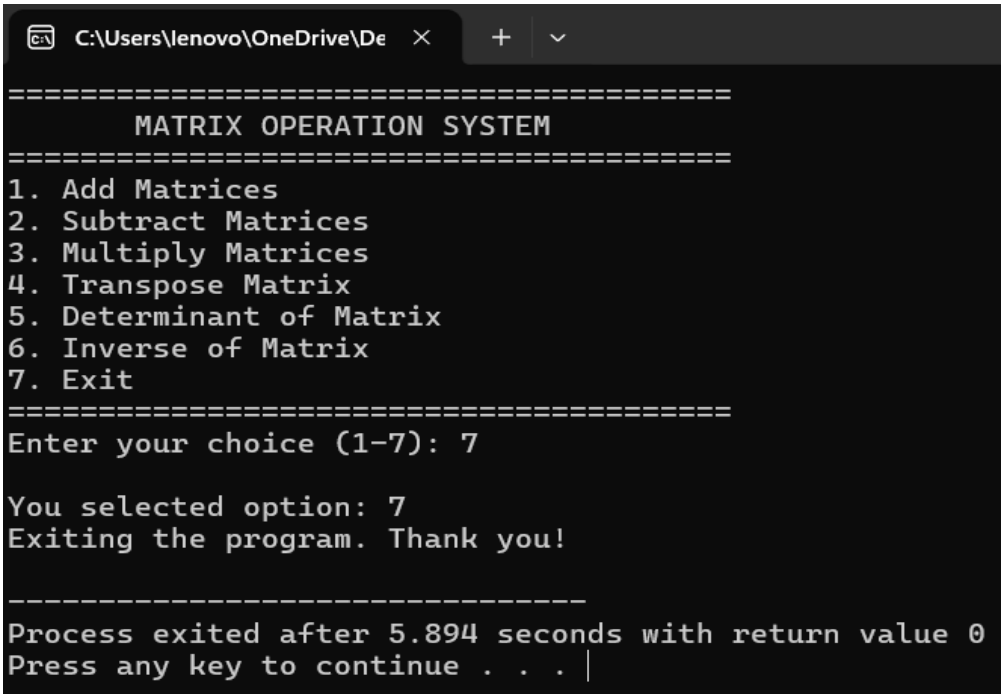


Fig 2.2: Matrix Input

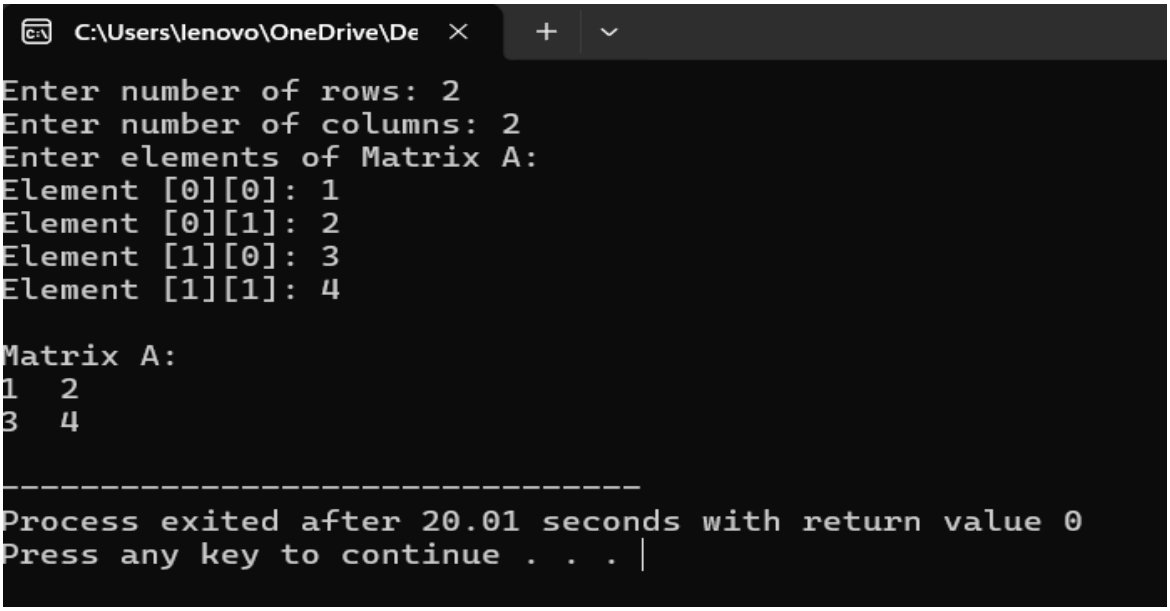


Fig 2.3: Matrix Addition Output

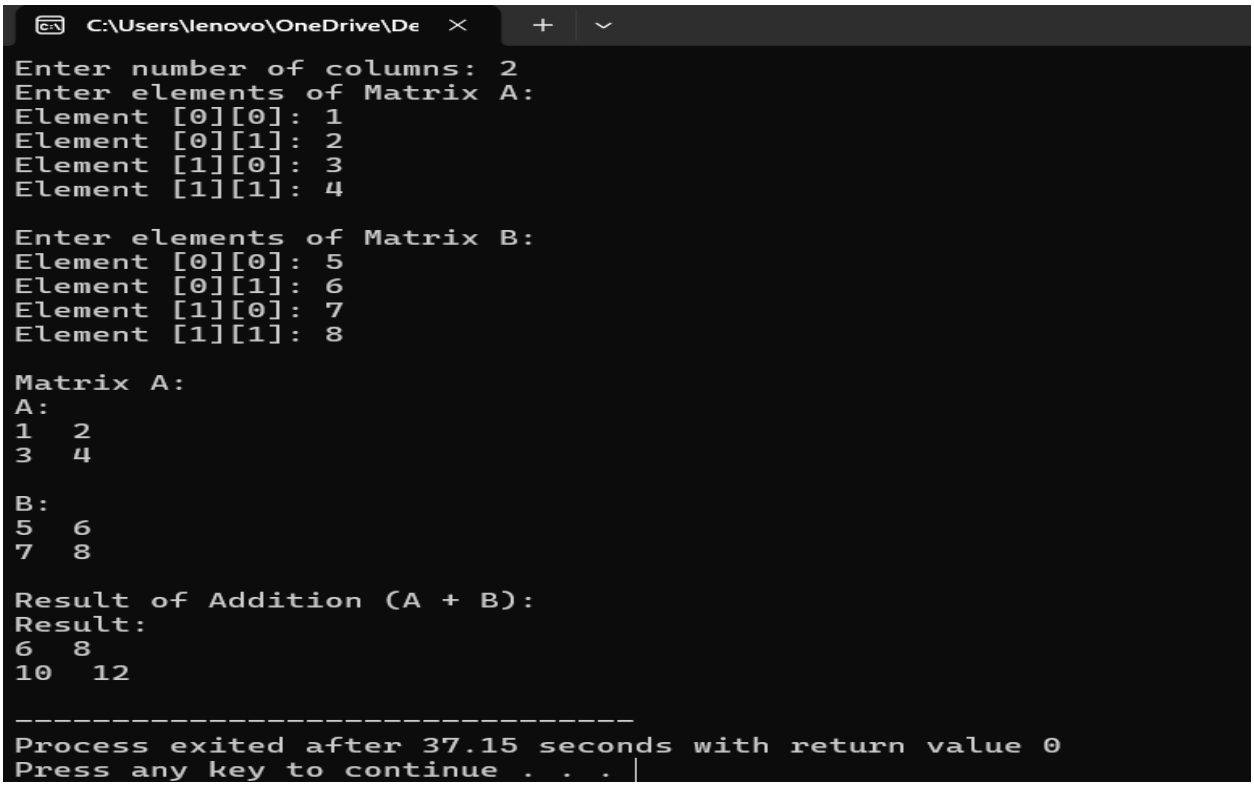


Fig 2.4: Matrix Multiplication Output

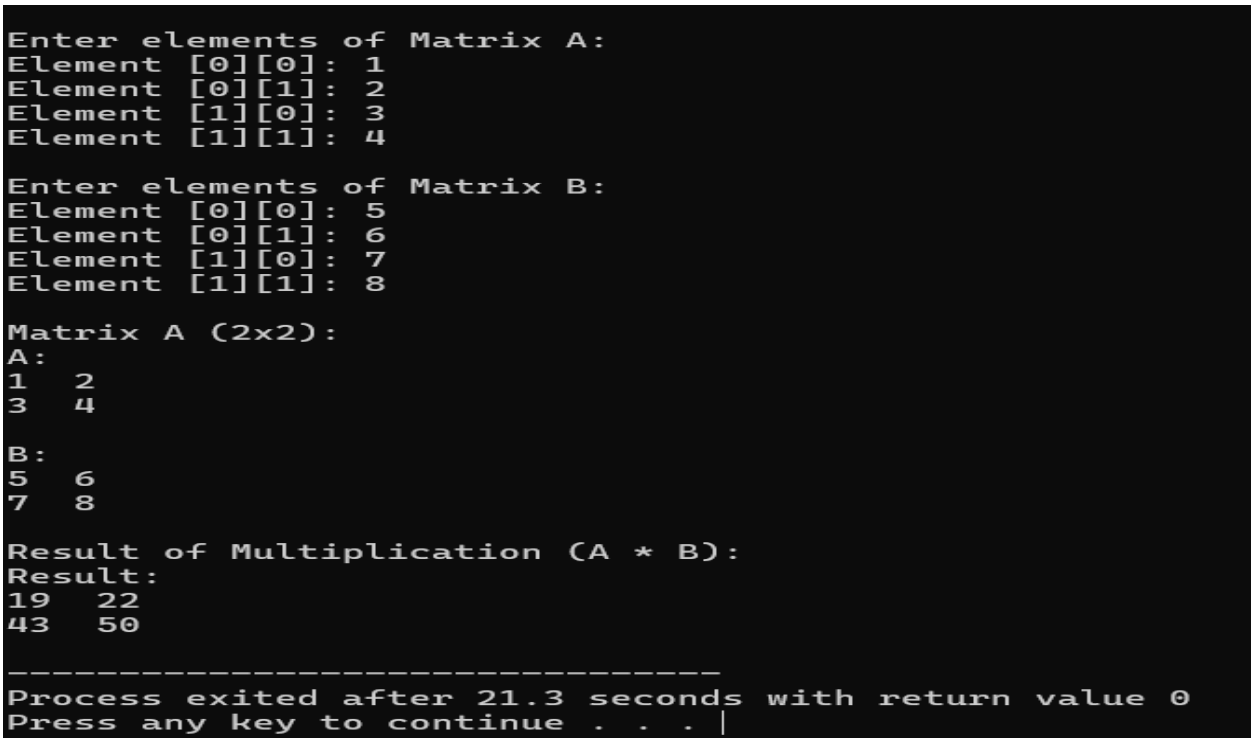


Fig 2.5: Matrix Transpose Output

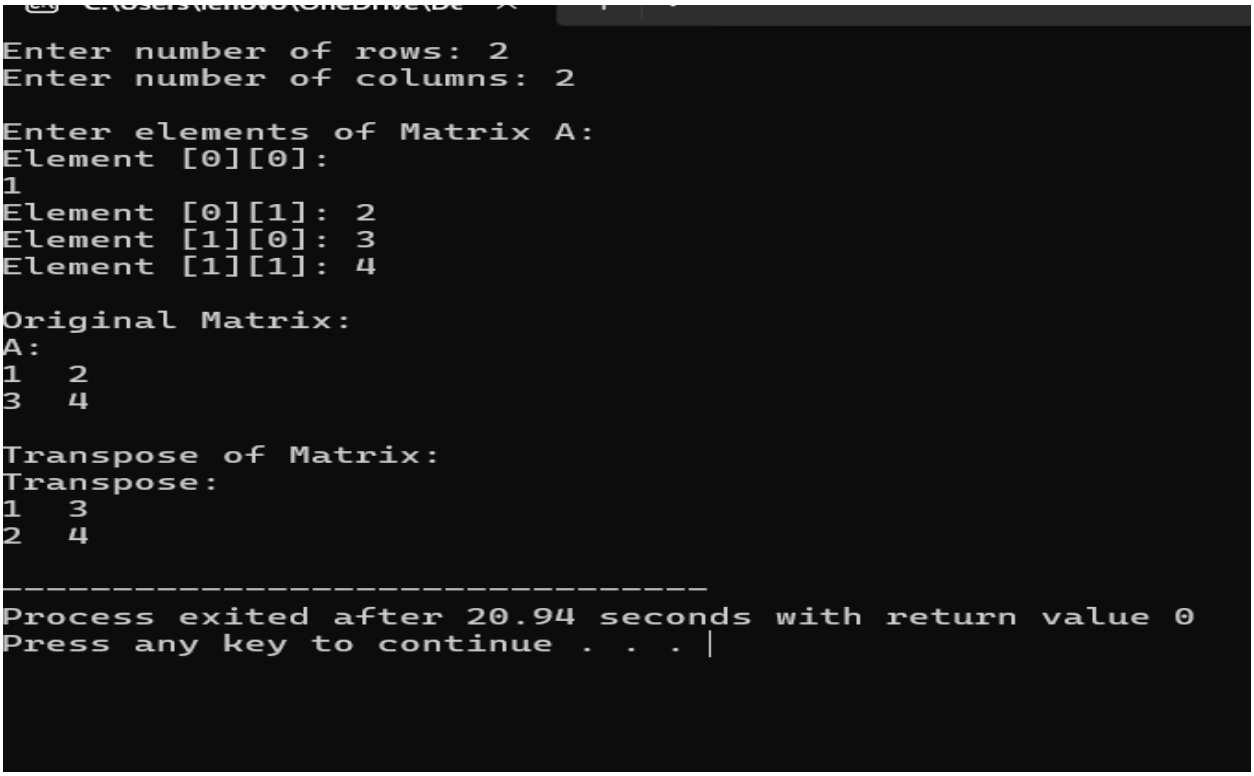


Fig 2.6: Determinant Calculation Output

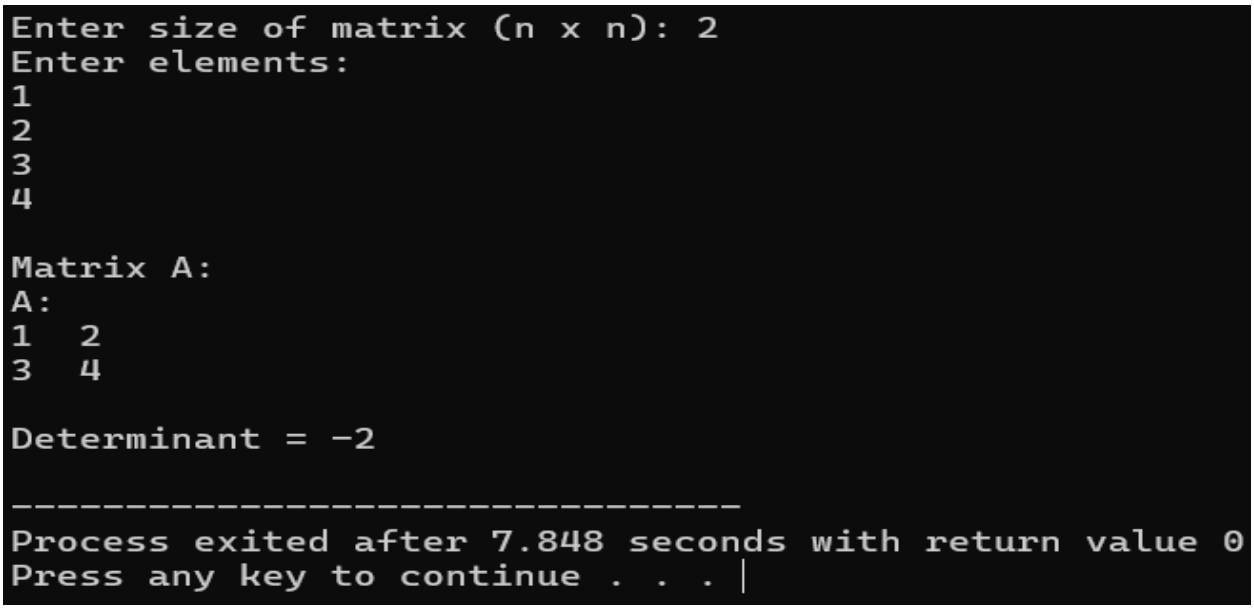


Fig 2.7: Inverse Calculation Output

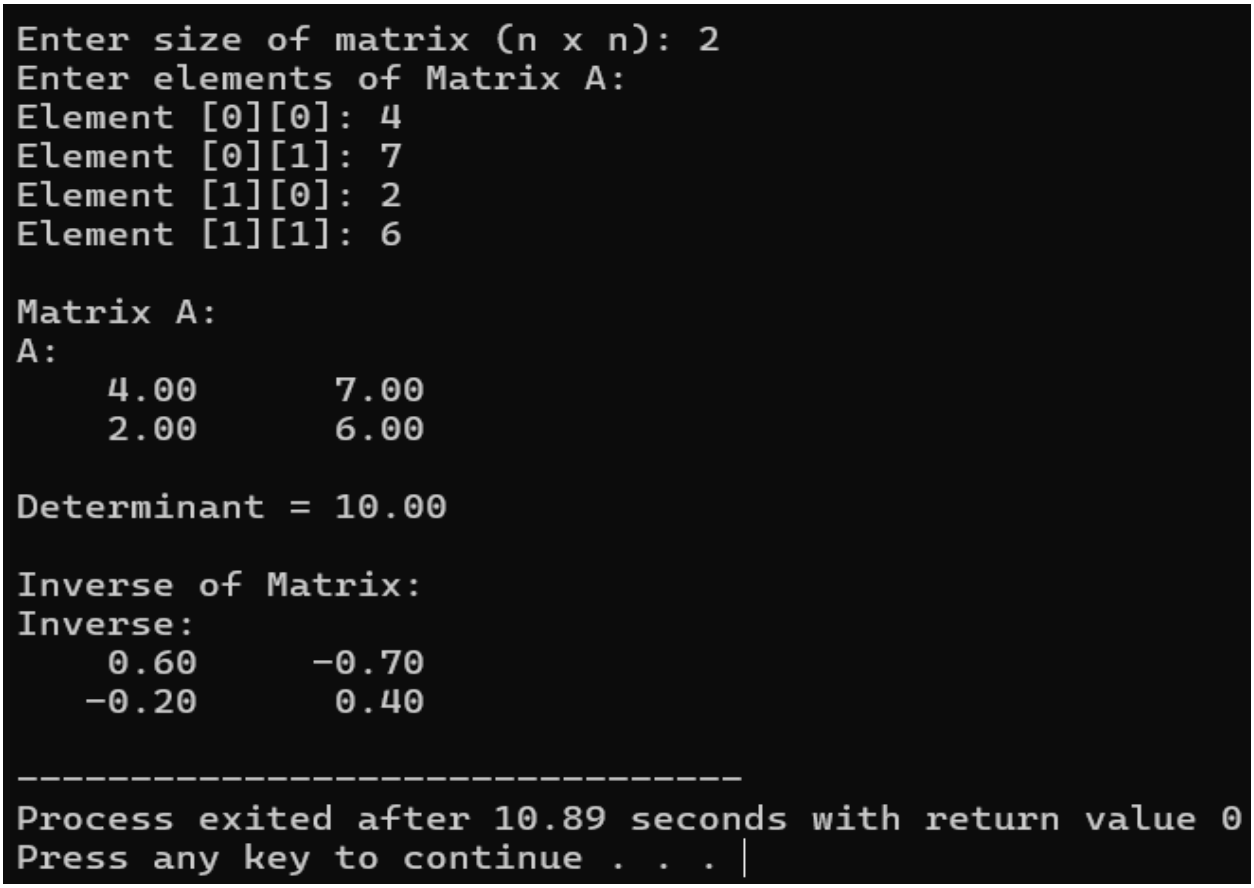


Fig 2.8: Validation Error Message

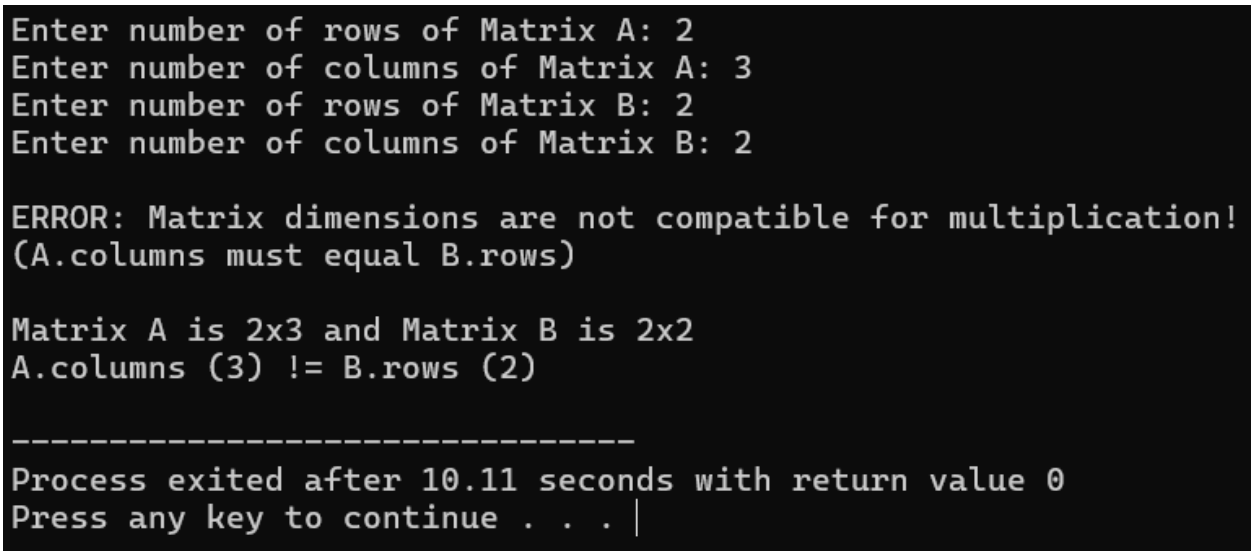
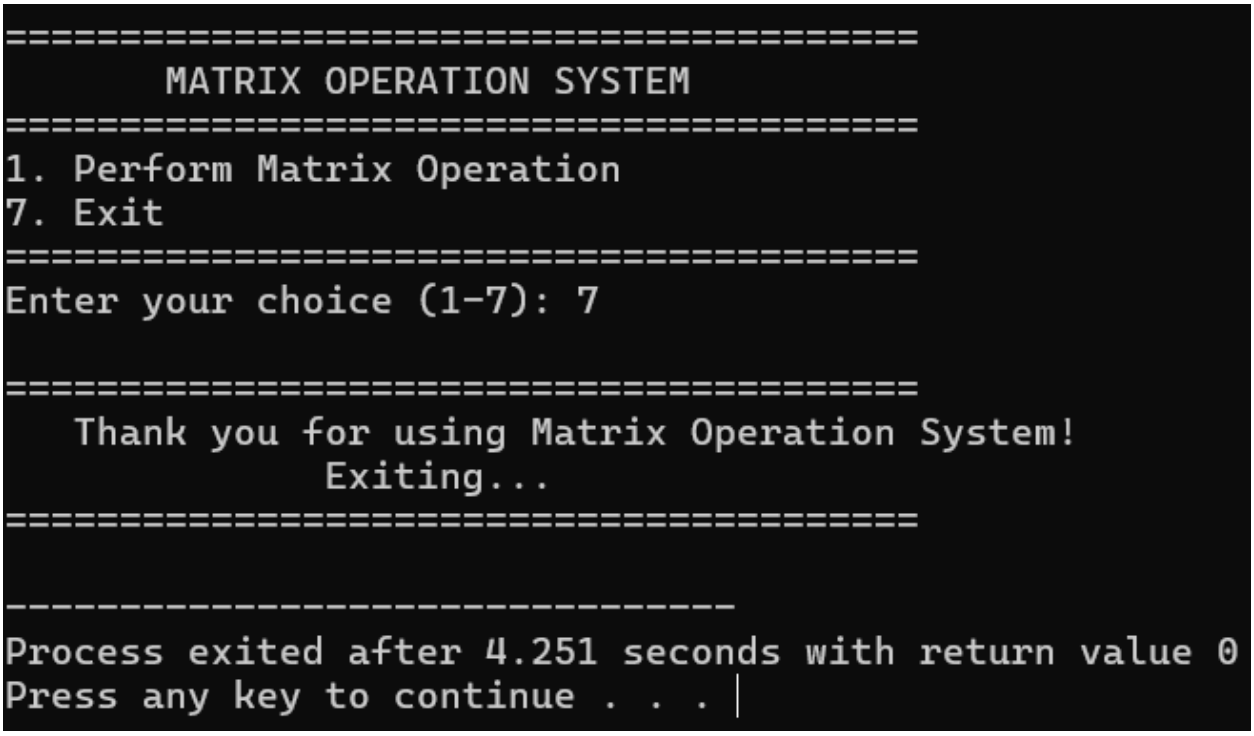


Fig 2.9: Exit Message



CHAPTER -3 Conclusion & Future Scope

3.1. Conclusion

The Matrix Operation System successfully demonstrates the application of Object-Oriented Programming principles in C++ to solve a real-world mathematical problem. The project provides a functional, menu-driven interface that accurately performs matrix addition, subtraction, multiplication, transpose, determinant, and inverse operations. By incorporating dimension validation, the system effectively reduces manual errors and ensures mathematical correctness. The modular design, based on a clear three-layer architecture, makes the code reusable, maintainable, and easy to extend. The project achieves its objectives of reducing manual effort, applying OOP concepts, and delivering a reliable tool for students, faculty, and developers.

3.2. Future Scope

The current system provides a solid foundation and can be extended in several ways:

- **Graphical User Interface (GUI):** A GUI can be developed using frameworks like Qt or Dear ImGui to make the system more visually appealing and user-friendly.
- **File Storage:** Functionality to save and load matrices from files can be added for persistent data storage and reuse.
- **Additional Operations:** More advanced matrix operations such as rank, eigenvalues, eigenvectors, and solving systems of linear equations can be implemented.
- **Support for Different Data Types:** The system can be extended to support matrices of floating-point numbers or complex numbers.
- **Performance Optimization:** For large matrices, optimized algorithms (e.g., Strassen's algorithm for multiplication) can be integrated to improve performance.
- **Web or Mobile Application:** The logic can be ported to a web or mobile platform for broader accessibility.

THANK YOU